

CS 316: Introduction to Deep Learning

Multi Layer Perceptron Week 7

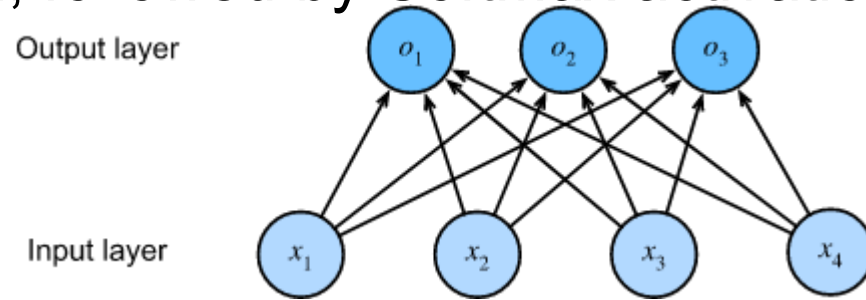
Dr Abdul Samad

Lecture Outline

- Affine Transformation
- Limitation of Linear Models
- Hidden Layers
- Linearity to Non-Linearity
- Activation Functions
 - ReLU
 - Sigmoid
 - Tanh

Affine Transformation

- In SoftMax regression, inputs are directly mapped to the outputs by a single affine transformation, followed by SoftMax activation.



- If the labels are truly related to the input data via an affine transformation, this approach is sufficient.
- However, linearity in affine transformations is a strong *assumption*.

Limitation of Linear Models

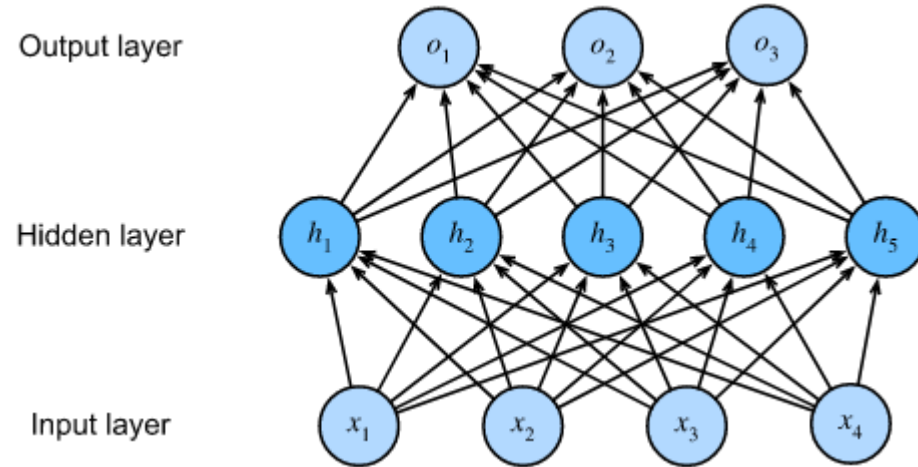
- Linearity implies the weaker assumption of monotonicity i.e., any increase in our feature must either always cause an increase in our model's output, or always cause a decrease in our model's output.
- For example, when predicting whether a person would repay a loan, we could expect that an applicant with higher income would always be more likely to repay than one with a lower income.
- The relationship is monotonic but not linear. One way to make linearity more plausible is to preprocess the data i.e., take logarithm of income as a feature.
- However, when classifying images of cats and dogs, the issue cannot be resolved by a simple preprocessing fix.
- We can handle these limitations of linear model by incorporating one or more hidden layers.

Incorporating Hidden Layers

- We can use deep neural networks to jointly learn a representation via hidden layers and a linear predictor that acts on that representation using observational data.
- The easiest way to incorporate hidden layers is to stack many fully-connected layers on top of each other.
- Each layer feeds into the layer above it, until we generate outputs.
- The first $L - 1$ layer's act as our representation, and the final layer as our linear predictor.
- This architecture is called Multi-Layer Perceptron (MLP)

Multi Layer Perceptron - Example

- The MLP has 4 inputs, 3 outputs, and its hidden layer contains 5 hidden units.



- The input layer does not involve any calculations, producing outputs with this network requires computations for both the hidden and output layers. Thus, the number of layers
- Every input influences every neuron in the hidden layer, and each of these in turn influences every neuron in the output layer.

Figure taken from [Multi Layer Perceptron](#)

Equivalence of Linear Models

- $\mathbf{X} \in \mathbb{R}^{n \times d}$ where n is number of examples and d is the number of features.
- The output of one-hidden layer of MLP is denoted by $\mathbf{H} \in \mathbb{R}^{n \times h}$ where h represents the number of hidden units.
- The hidden layer parameters are defined as $\mathbf{W}^{(1)} \in \mathbb{R}^{d \times h}$ and $\mathbf{b}^{(1)} \in \mathbb{R}^{1 \times h}$
- The output layer parameters are defined as $\mathbf{W}^{(2)} \in \mathbb{R}^{h \times q}$ and $\mathbf{b}^{(2)} \in \mathbb{R}^{1 \times q}$ where q represents the number of output units.
- The output of one-hidden layer of MLP is denoted by $\mathbf{O} \in \mathbb{R}^{n \times q}$.

$$\mathbf{H} = \mathbf{XW}^{(1)} + \mathbf{b}^{(1)}$$

$$\mathbf{O} = \mathbf{HW}^{(2)} + \mathbf{b}^{(2)}$$

- The hidden units are given by an affine function of the input, and the output is given by an affine function of the hidden units. An affine function of an affine function is itself an affine function.
- Consequently, we can remove the hidden layer and obtain an equivalent model with parameters $\mathbf{W} = \mathbf{W}^{(1)}\mathbf{W}^{(2)}$ and $\mathbf{b} = \mathbf{b}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}$

Equivalence of Linear Models - Example

Prove that a one-hidden layer model with parameters $\mathbf{W} = \mathbf{W}^{(1)}\mathbf{W}^{(2)}$ and $\mathbf{b} = \mathbf{b}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}$ is equivalent to a single-layer model.

$$\mathbf{H} = \mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)}$$

$$\mathbf{O} = \mathbf{H}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}$$

$$\mathbf{O} = (\mathbf{X}\mathbf{W}^{(1)} + \mathbf{b}^{(1)})\mathbf{W}^{(2)} + \mathbf{b}^{(2)}$$

$$\mathbf{O} = \mathbf{X}\mathbf{W}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)}$$

$$\mathbf{O} = \mathbf{X}(\mathbf{W}^{(1)}\mathbf{W}^{(2)}) + (\mathbf{b}^{(1)}\mathbf{W}^{(2)} + \mathbf{b}^{(2)})$$

$$\mathbf{O} = \mathbf{X}\mathbf{W} + \mathbf{b}$$

Activation Functions

- In order to realize the potential of MLP, a nonlinear activation function is to be applied to each hidden unit following the affine transformation.
- The output of activation functions are called activations.
- In general, with the activation functions in place, it is no longer possible to collapse our MLP into a linear model:

$$\begin{aligned} \mathbf{H} &= \sigma(\mathbf{XW}^{(1)} + \mathbf{b}^{(1)}) \\ \mathbf{O} &= \mathbf{HW}^{(2)} + \mathbf{b}^{(2)} \end{aligned}$$

- Activation functions decide whether a neuron should be activated or not by calculating the weighted sum and further adding bias with it.
- They are differentiable operators to transform input signals to outputs, while most of them add non-linearity.

Activation Functions - ReLU

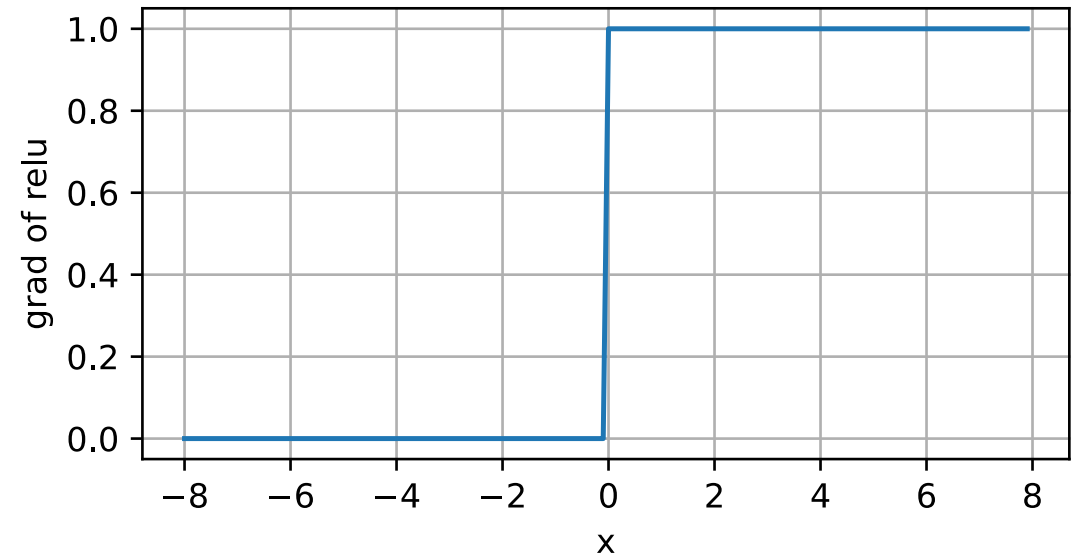
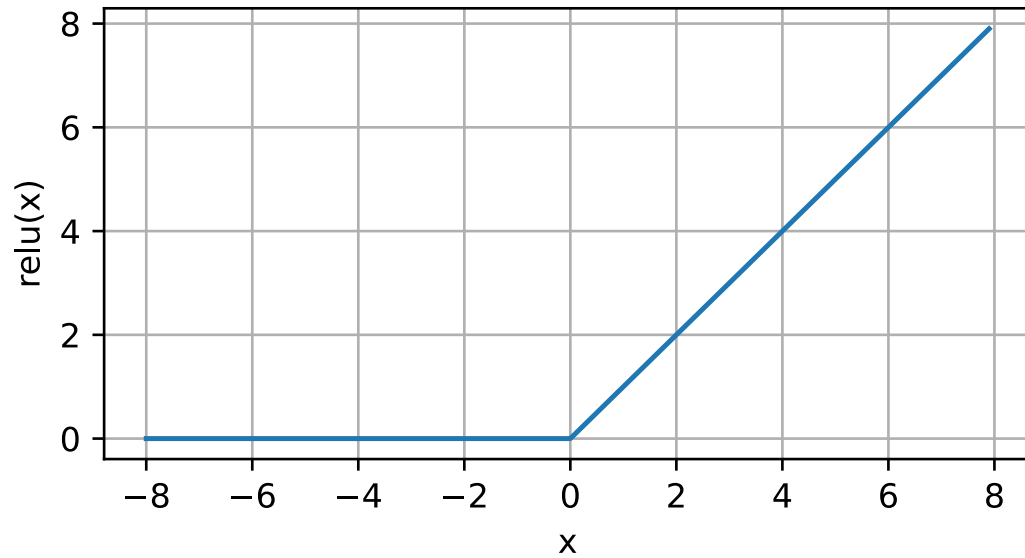
- ReLU stands for Rectified Linear Unit.
- Given an element x , the function is defined as the maximum of that element and 0.

$$\text{ReLU}(x) = \max(x, 0)$$

- Informally, the ReLU function retains only positive elements and discards all negative element by setting the corresponding activations to 0.
- When the input is negative, the derivative of the ReLU is 0, and when the input is positive, the derivative of the ReLU function is 1.

Activation Functions - ReLU

The following is the graph of the ReLU function and its derivative:



Figures taken from [ReLU](#), [Grad ReLU](#)

Activation Functions - Sigmoid

- The sigmoid function transforms its inputs, for which values lie in the domain $\mathbb{R}$, to outputs that lie on the interval $(0,1)$.
- The sigmoid function is defined as follows:

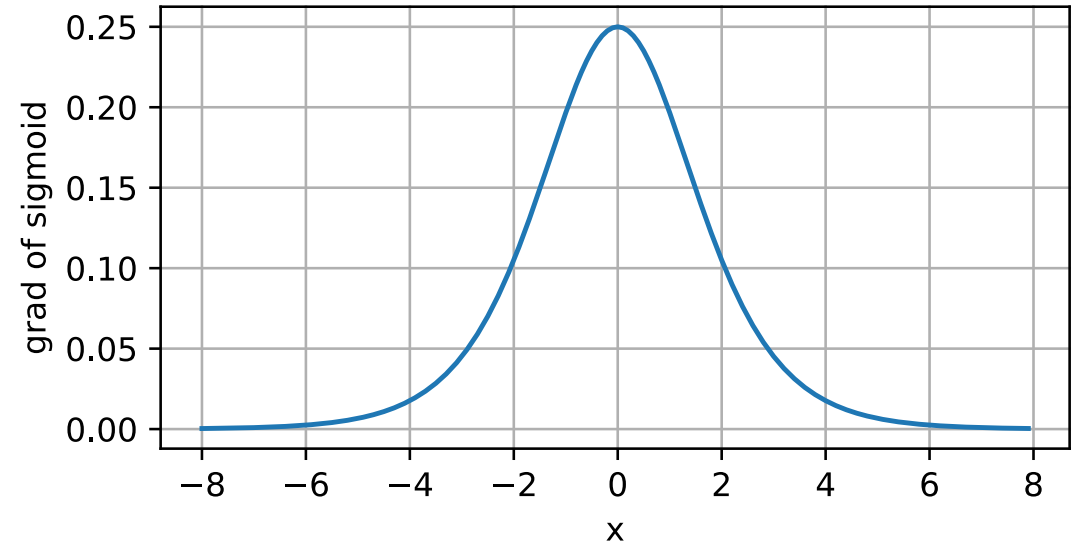
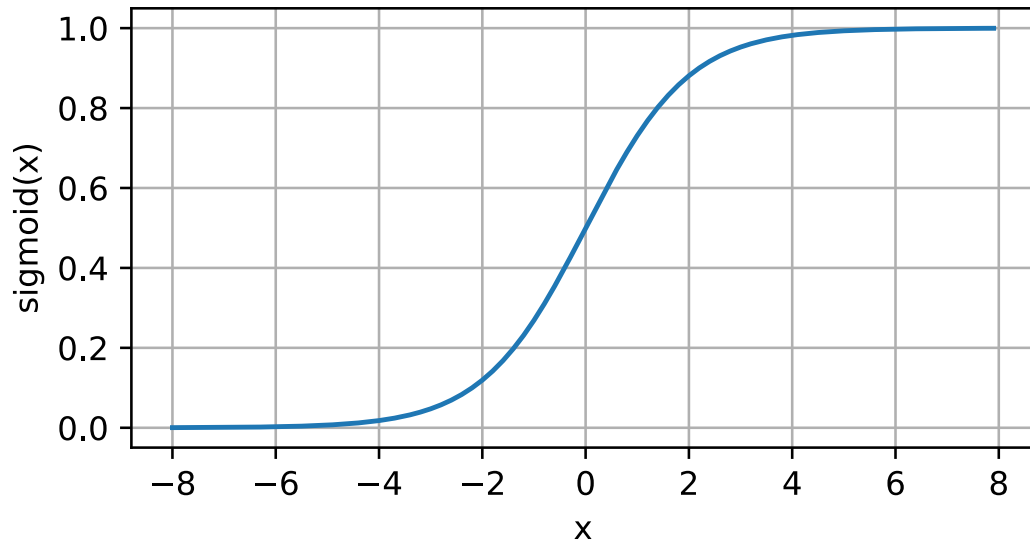
$$\textit{sigmoid}(x) = \frac{1}{1 + \exp(-x)}$$

- The sigmoid is often called a squashing function: it squashes input in the range $(-\infty, +\infty)$ to some value in the range $(0,1)$.
- The derivative of the sigmoid function is defined as follows:

$$\frac{d[\textit{sigmoid}(x)]}{dx} = \frac{\exp(-x)}{(1 + \exp(-x))^2} = \textit{sigmoid}(x)(1 - \textit{sigmoid}(x))$$

Activation Functions - Sigmoid

The following is the graph of the Sigmoid function and its derivative:



Figures taken from [Sigmoid](#), [Grad Sigmoid](#)

Activation Functions - Tanh

- The tanh transforms its inputs, for which values lie in the domain $\mathbb{R}$, to outputs that lie on the interval $(-1, +1)$.
- The tanh function is defined as follows:

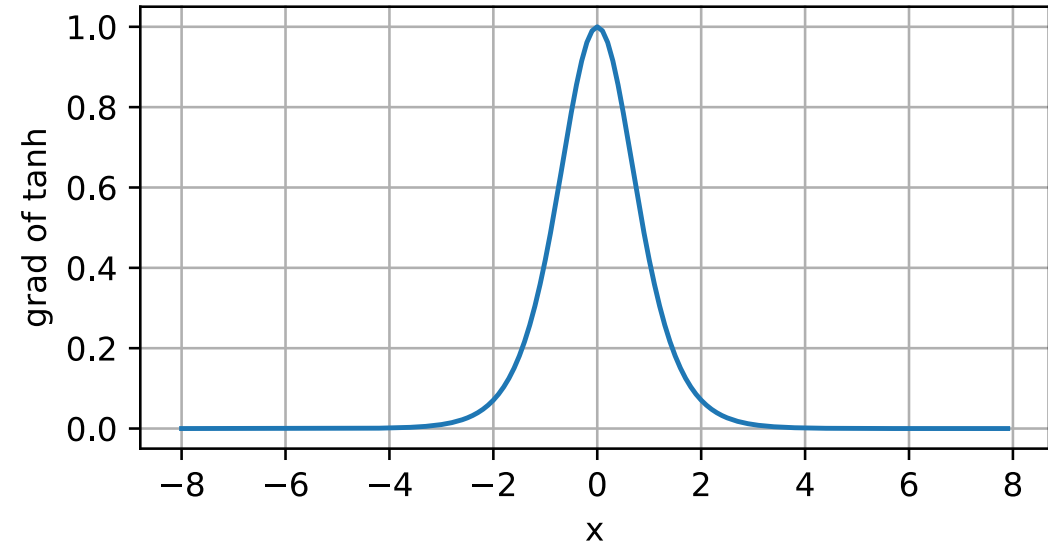
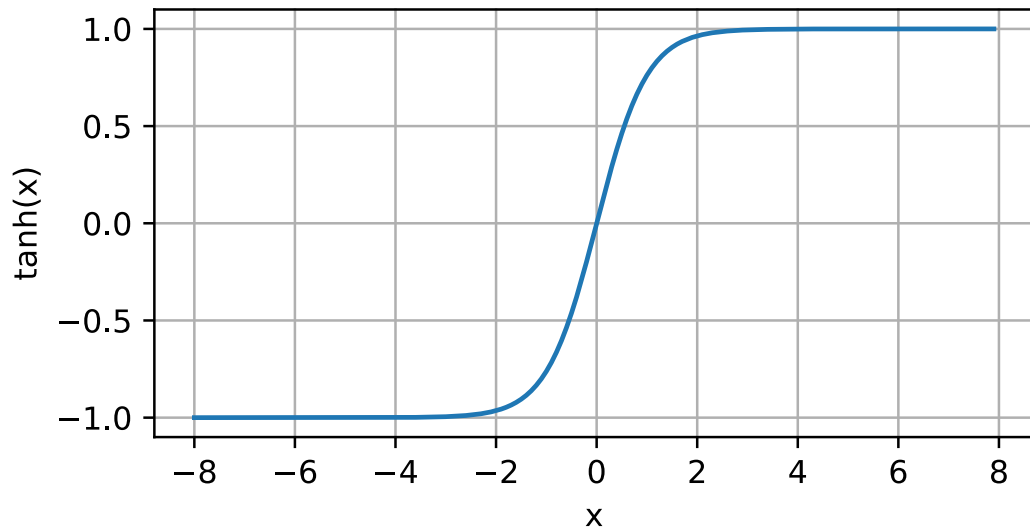
$$\tanh(x) = \frac{1 - \exp(-2x)}{1 + \exp(-2x)}$$

- The derivative of the tanh function is defined as follows:

$$\frac{d[\tanh(x)]}{dx} = 1 - \tanh^2(x)$$

Activation Functions - Tanh

The following is the graph of the Tanh function and its derivative:



Figures taken from [Tanh](#), [Grad Tanh](#)

References

- https://d2l.ai/chapter_multilayer-perceptrons/index.html

CS 316: Introduction to Deep Learning

Forward Propagation, Backward
Propagation, and Computational
Graphs

Week 7

Dr Abdul Samad

Lecture Outline

- Forward Propagation
- Computational Graphs
- Backward Propagation

Forward Propagation

- Refers to the calculation and storage of intermediate variables (including outputs) for a neural network in order from the input layer to the output layer.

Forward Propagation in a MLP with one hidden layer

- Assume the input is $\mathbf{x} \in \mathbb{R}^d$ and the hidden layer does not include a bias term. The intermediate variable $\mathbf{z}$ is then computed as follows:

$$\mathbf{z} = \mathbf{W}^{(1)}\mathbf{x}$$

- $\mathbf{W}^{(1)} \in \mathbb{R}^{h \times d}$ represents the weights of the hidden layer. The intermediate variable $\mathbf{z}$ is then passed as a input to the activation function ϕ to compute the output of the hidden layer which is represented by $\mathbf{h}$

$$\mathbf{h} = \phi(\mathbf{z})$$

- The output of the final layer represented by $\mathbf{o}$ is then computed as follows:

$$\mathbf{o} = \mathbf{W}^{(2)}\mathbf{h}$$

- $\mathbf{W}^{(2)} \in \mathbb{R}^{q \times h}$ represents the weights of the output layer.

Forward Propagation in a MLP with one hidden layer

- Given the loss function l and example label y , the loss for a single data point is computed as follows:

$$L = l(\mathbf{o}, y)$$

- The l_2 regularization term given the hyperparameter λ is computed as follows:

$$s = \frac{\lambda}{2} \left(\|\mathbf{W}^{(1)}\|_F^2 + \|\mathbf{W}^{(2)}\|_F^2 \right)$$

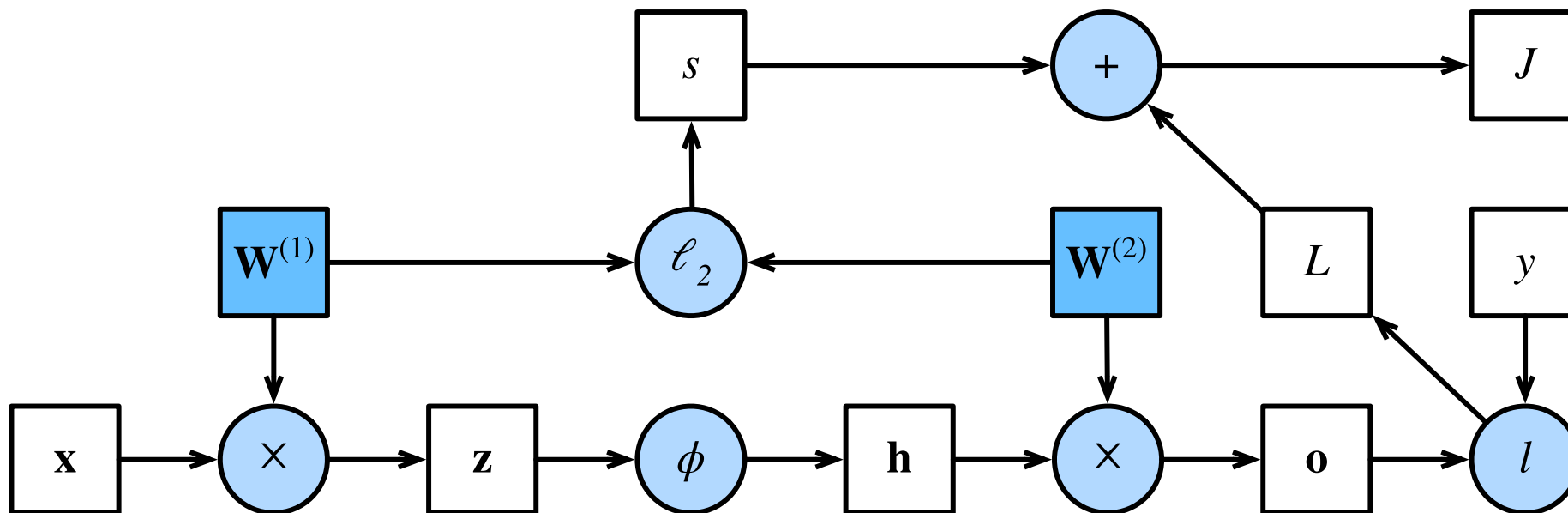
- The model's regularized loss is then computed as follows:

$$J = L + s$$

Computational Graphs

- Computational graphs helps us visualize the dependencies of operators and variables within the calculation.
- The squares denotes the variables and circles denotes the operators. The lower left corner indicates the input and upper right corner indicates the output.
- The directions of the arrows (which illustrate data flow) are primarily rightward and upward.

Computational Graph of MLP with one hidden layer



Backward Propagation

- Backpropagation refers to the method of calculating the gradient of parameters of the neural network with respect to the loss function.
- The method traverses the network in reverse order, from the output of the input layer, according to the chain rule from calculus.
- The algorithm stores any intermediate variables required while calculating the gradient with respect to some parameters.

Backward Propagation using Chain Rule

- Assume that we functions $Y = f(X)$ and $Z = g(Y)$ in which the input and output X, Y, Z are tensors of arbitrary shapes.
- By using the chain rule, we can compute the derivative of Z with respect to X as follows:

$$\frac{\delta Z}{\delta X} = \text{prod}\left(\frac{\delta Z}{\delta Y}, \frac{\delta Y}{\delta X}\right)$$

Backward Propagation in a MLP with one hidden layer

- The first step is to calculate the gradient of the objective function J with respect to the loss term L and the regularization term s

$$\frac{\delta J}{\delta L} = 1 \quad \frac{\delta J}{\delta s} = 1$$

- The gradient of the output $\mathbf{o}$ with respect to the objective function J is computed as follows:

$$\frac{\delta J}{\delta \mathbf{o}} = \text{prod}\left(\frac{\delta J}{\delta L}, \frac{\delta L}{\delta \mathbf{o}}\right) = \frac{\delta L}{\delta \mathbf{o}}$$

- The gradient of the regularization term s with respect to weights is computed as follows:

$$\frac{\delta s}{\delta \mathbf{W}^{(1)}} = \lambda \mathbf{W}^{(1)} \quad \frac{\delta s}{\delta \mathbf{W}^{(2)}} = \lambda \mathbf{W}^{(2)}$$

Backward Propagation in a MLP with one hidden layer

- The gradient of the weights of the output layer $\mathbf{W}^{(2)}$ with respect to the objective function is computed as follows:

$$\frac{\delta J}{\delta \mathbf{W}^{(2)}} = prod\left(\frac{\delta J}{\delta o}, \frac{\delta o}{\delta \mathbf{W}^{(2)}}\right) + prod\left(\frac{\delta J}{\delta s}, \frac{\delta s}{\delta \mathbf{W}^{(2)}}\right) = \frac{\delta J}{\delta o} \mathbf{h}^T + \lambda \mathbf{W}^{(2)}$$

- The gradient with respect to the output of the hidden layer $\mathbf{h}$ is computed as follows:

$$\frac{\delta J}{\delta \mathbf{h}} = prod\left(\frac{\delta J}{\delta o}, \frac{\delta o}{\delta \mathbf{h}}\right) = \mathbf{W}^{(2)T} \frac{\delta J}{\delta o}$$

- The gradient with respect to the intermediate variable $\mathbf{z}$ is computed as follows:

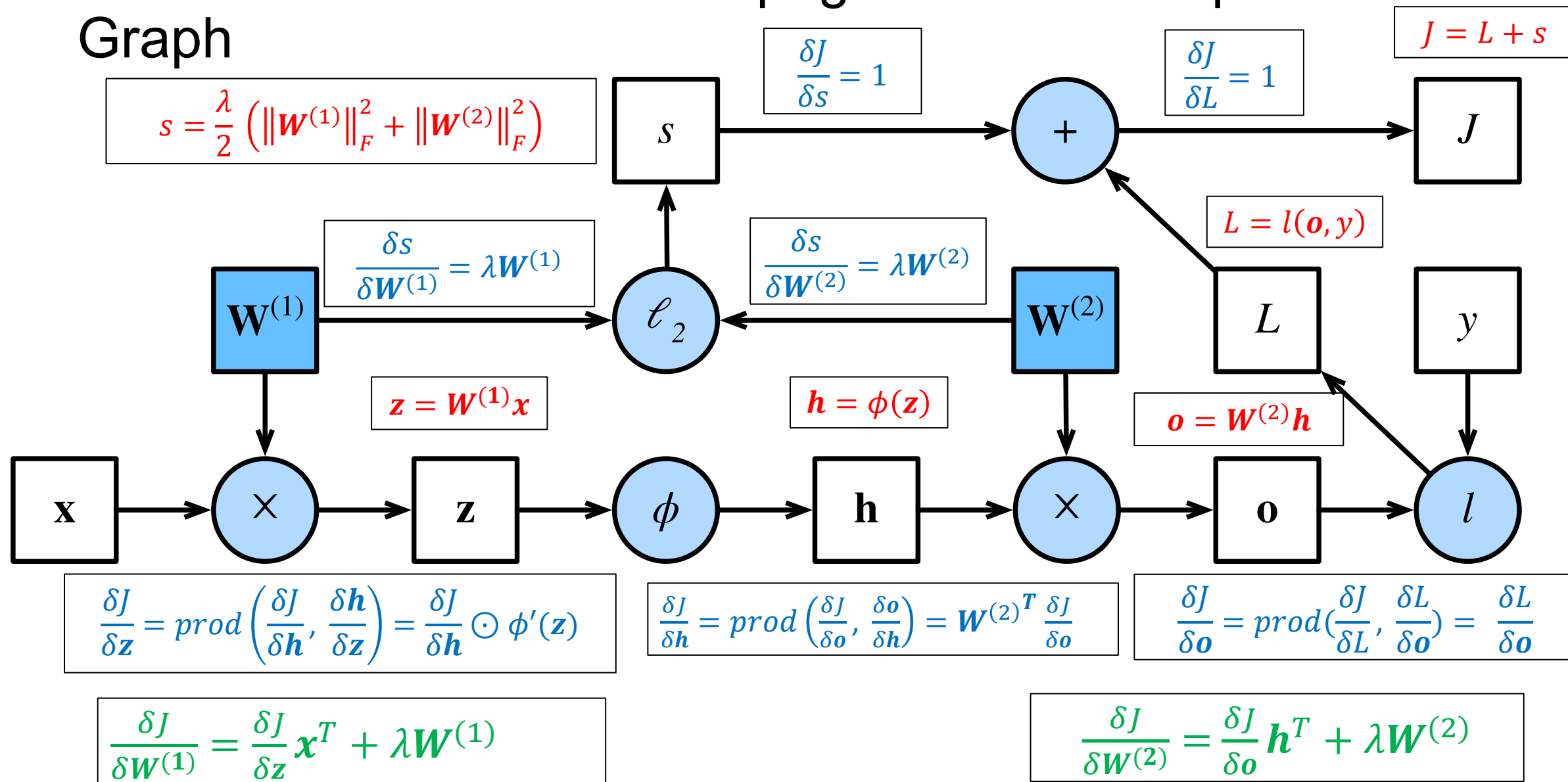
$$\frac{\delta J}{\delta \mathbf{z}} = prod\left(\frac{\delta J}{\delta \mathbf{h}}, \frac{\delta \mathbf{h}}{\delta \mathbf{z}}\right) = \frac{\delta J}{\delta \mathbf{h}} \odot \phi'(\mathbf{z})$$

Backward Propagation in a MLP with one hidden layer

- Finally, the gradient of the weights of the hidden layer $\mathbf{W}^{(1)}$ with respect to the objective function is computed as follows:

$$\frac{\delta J}{\delta \mathbf{W}^{(1)}} = \text{prod} \left(\frac{\delta J}{\delta \mathbf{z}}, \frac{\delta \mathbf{z}}{\delta \mathbf{W}^{(1)}} \right) + \text{prod} \left(\frac{\delta J}{\delta s}, \frac{\delta s}{\delta \mathbf{W}^{(1)}} \right) = \frac{\delta J}{\delta \mathbf{z}} \mathbf{x}^T + \lambda \mathbf{W}^{(1)}$$

Forward and Backward Propagation in a Computational Graph



References

- https://d2l.ai/chapter_multilayer-perceptrons/backprop.html#